
Understanding Transformers

A Deep Dive into the Architecture That
Revolutionized Deep Learning

Project Report

*Submitted in partial fulfillment of the requirements for the
Artificial Neural Networks (ANN) Course*

Spring Semester 2026

Presented by:

Serag Khaled

Hussein Mostafa

Mohannad Hassan

Supervised by:

Dr. Mohsen Rashwan & Dr. Mohamed Abdelghany

Interactive Demo: <https://transformers-ann-project.netlify.app/>

Submission Date: May 2026

Contents

1	Introduction and Project Aim	3
1.1	Project Aim	3
2	Background: Why Transformers?	3
2.1	Limitations of Prior Architectures	3
2.2	The Transformer Solution	4
3	Architecture Overview	4
3.1	Key Components	4
3.1.1	Token Embeddings	4
3.1.2	Positional Encoding	5
3.1.3	Multi-Head Self-Attention	5
3.1.4	Feed-Forward Network (FFN)	5
3.1.5	Add & Layer Normalization	5
3.1.6	Masked Multi-Head Attention (Decoder)	5
4	Project Deliverables	6
4.1	Deliverable 1: Presentation Slides	6
4.2	Deliverable 2: Interactive Website / Demo	7
4.3	Deliverable 3: MCQ Question Bank	7
5	Steps and Methodology	8
5.1	Block Diagram of the Project Workflow	8
5.2	Step-by-Step Description	8
6	Challenges Faced and How We Managed Them	9
6.1	Challenge 1: Depth vs. Accessibility Trade-off	9
6.2	Challenge 2: Verifying AI-Generated Content	10
6.3	Challenge 3: Making Self-Attention Interactive	10
6.4	Challenge 4: Distinguishing Encoder-Only, Decoder-Only, and Encoder-Decoder Models	10
6.5	Challenge 5: Designing Meaningful MCQs at Multiple Levels	10
6.6	Challenge 6: Lack of Web Development and UI Design Knowledge	11
6.7	Challenge 7: Maintaining Pedagogical Coherence Across Multiple Deliverables	11
6.8	Challenge 8: Controlling the Tone and Register of AI-Generated Explanations	11
6.9	Challenge 9: Avoiding Hallucinated Citations and Fabricated Claims	12

7	AI Tools Usage Documentation	12
8	Conclusions	13

1. Introduction and Project Aim

This report documents the design, development, and reflection process for the ANN Course Project. Our team selected **Transformer Neural Networks** as the concept to explain, motivated by their central role in modern deep learning. Transformers underpin virtually every state-of-the-art model in Natural Language Processing (NLP), computer vision, and multimodal AI—from GPT and BERT to Claude and Gemini.

1.1. Project Aim

The aim of this project is threefold:

1. **Conceptual Mastery:** Develop and demonstrate a thorough, accurate understanding of the Transformer architecture, including its motivations, components, mathematical foundations, and limitations.
2. **Effective Communication:** Translate this technical knowledge into clear, visually compelling, and accessible educational materials—namely a presentation deck, an interactive website, and a question bank.
3. **Responsible AI Usage:** Use AI tools as *assistants* to accelerate content creation and ideation, while ensuring that all generated material is understood, verified, and curated by the team.

Core Concept

A **Transformer** is a deep learning architecture introduced by Vaswani et al. in the landmark 2017 paper “*Attention Is All You Need*.” It replaces sequential recurrence (used by RNNs and LSTMs) with a **self-attention mechanism** that allows every token in a sequence to directly attend to every other token simultaneously—enabling parallelization, capturing long-range dependencies, and scaling to billions of parameters.

2. Background: Why Transformers?

2.1. Limitations of Prior Architectures

Before Transformers, Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks were the dominant approach to sequence modeling. However, they suffer from several fundamental limitations:

- **Sequential Processing:** Tokens must be processed one at a time, preventing parallelization and limiting training speed on modern hardware.

- **Vanishing Gradients:** Information about distant tokens fades during back-propagation through time, making it difficult to learn long-range dependencies.
- **Memory Inefficiency:** Maintaining hidden states across long sequences requires significant memory and leads to information loss.
- **Scalability Ceiling:** RNNs do not benefit proportionally from increased data or compute.

2.2. The Transformer Solution

Transformers address each of these limitations directly:

Table 1: Comparison of RNN/LSTM vs. Transformer properties

Property	RNN / LSTM	Transformer
Processing	Sequential	Fully Parallel
Long-range dependencies	Difficult (vanishing gradients)	Direct (via self-attention)
Training speed	Slow	Fast
Memory with sequence length	$O(n)$	$O(n^2)$ (attention matrix)
Scalability	Limited	Highly scalable
Domain versatility	Primarily sequential	NLP, vision, speech, code, and more

3. Architecture Overview

The original Transformer follows an **encoder–decoder** structure. Both the encoder and decoder are stacks of $N = 6$ identical layers (in the original paper). The overall pipeline is:

Input Sequence $\rightarrow$ Embeddings + Positional Encoding $\rightarrow$ $\underbrace{\text{Encoder Stack}}_{\times N} \rightarrow \underbrace{\text{Decoder Stack}}_{\times N} \rightarrow$ Linear + Softmax $\rightarrow$ Output Probabilities

3.1. Key Components

3.1.1. Token Embeddings

Each input token is mapped to a dense vector of dimension d_{model} (typically 512). These vectors capture semantic meaning.

3.1.2. Positional Encoding

Since Transformers have no recurrence, they have no inherent notion of token order. Positional encodings are added to the token embeddings to inject sequence-position information. The original paper uses sinusoidal encodings:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

Learned (trainable) positional embeddings are also widely used in modern variants.

3.1.3. Multi-Head Self-Attention

This is the core innovation of the Transformer. For each token, three vectors are computed—**Query (Q)**, **Key (K)**, and **Value (V)**—by projecting the input with learned weight matrices. The attention output is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Multi-head attention runs h attention heads in parallel, each attending to a different learned subspace, and concatenates the results:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$.

3.1.4. Feed-Forward Network (FFN)

After attention, each token is passed independently through a two-layer fully connected network:

$$\text{FFN}(x) = \max(0, xW_1 + b_1) W_2 + b_2$$

This adds non-linearity and further transforms token representations.

3.1.5. Add & Layer Normalization

Residual (skip) connections are added around each sub-layer to preserve information and stabilize training. Layer normalization follows each residual connection:

$$\text{LayerNorm}(x + \text{Sublayer}(x))$$

3.1.6. Masked Multi-Head Attention (Decoder)

In the decoder, a **look-ahead mask** is applied to the attention scores so that each position can only attend to itself and previous positions—preventing the model from

“cheating” by looking at future tokens during training. The masking sets future positions to $-\infty$ before the softmax.

4. Project Deliverables

4.1. Deliverable 1: Presentation Slides

A 15-slide visual presentation was produced covering the following topics:

1. Title slide (cover)
2. Why Transformers over RNNs/LSTMs (motivation)
3. What are Transformers? (overview & architecture diagram)
4. Key components of the Transformer (magnified encoder view)
5. Embedding & Positional Embedding
6. Transformer Architecture: Encoder & Decoder (stacked layers)
7. Self-Attention Mechanism: core idea, key vectors, formula, and example
8. Self-Attention: How It Works (Q, K, V flow diagram)
9. Multi-Head Attention (parallel heads, concatenation, dimensions)
10. Masked Multi-Head Attention (look-ahead mask, decoder context)
11. Training & Inference of Transformers
12. Limitations & Failure Cases
13. Transformers Attention Quiz (5 MCQs with highlighted answers)
14. Call-to-action slide pointing to the interactive website
15. Thank You slide

The slides use a dark, tech-themed visual style with purple/blue gradients, consistent diagrams, and annotated mathematical formulas.

4.2. Deliverable 2: Interactive Website / Demo

An interactive educational website was deployed at:

<https://transformers-ann-project.netlify.app/>

The website includes five main sections:

- **Overview:** A narrative introduction to the Transformer, its historical context (from the 2017 “Attention Is All You Need” paper), and key innovations.
- **Architecture:** A clickable architecture diagram allowing users to explore each component (Input Embedding, Positional Encoding, Encoder, Decoder) with on-demand explanations.
- **Self-Attention Demo:** An animated visualization where users can click on individual words in a sample sentence and observe the attention weights connecting that word to all other tokens.
- **Interactive Pipeline:** A step-by-step text processing demo (Tokenization → Embedding → Positional Encoding → Self-Attention → Feed Forward → Output).
- **Quiz:** A 15-question multiple choice quiz with instant feedback and a running score.

4.3. Deliverable 3: MCQ Question Bank

A bank of **15 multiple-choice questions** was designed to cover all major aspects of the Transformer concept, including its motivation, architecture components, mathematical formulas, training procedure, and limitations. Each question has four options with clearly identified correct answers (examples were shown on the presentation’s quiz slide).

Sample questions include:

- What is the main purpose of the attention mechanism in Transformers?
- In self-attention, each token compares itself with which other tokens?
- Which components are used to compute attention scores?
- Why are Transformers faster to train than RNNs/LSTMs?
- What problem do Transformers handle better than traditional RNNs?

5. Steps and Methodology

5.1. Block Diagram of the Project Workflow

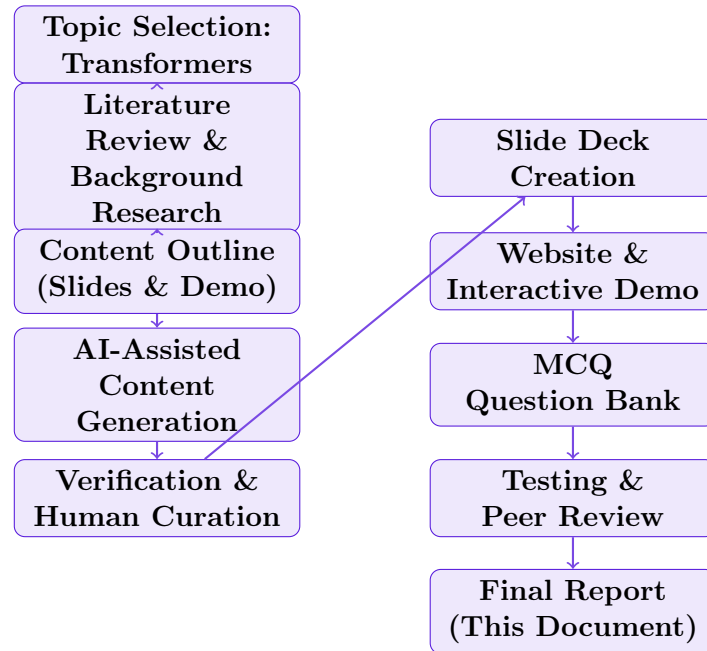


Figure 1: Block diagram of the project development workflow

5.2. Step-by-Step Description

Step 1: Topic Selection. The team selected Transformers as the project concept given their profound impact on modern AI and the richness of the material for both presentation and interaction.

Step 2: Literature Review. We studied the original 2017 paper “Attention Is All You Need” (Vaswani et al.), the Illustrated Transformer blog by Jay Alammar, and the Hugging Face documentation to build a solid technical foundation.

Step 3: Content Outline. We mapped out a logical pedagogical flow: starting with motivation (why not RNNs?), then architecture, then mathematical details (attention formula, multi-head attention, masking), then training/inference, then limitations.

Step 4: AI-Assisted Generation. Large language models (LLMs) and design tools were used to:

- Draft initial explanations of complex concepts in clear language.
- Generate initial code for the interactive website components.

- Suggest diagram layouts and visual metaphors.
- Produce an initial pool of MCQ questions.

Step 5: Verification and Curation. Each AI-generated output was critically reviewed by all team members. Mathematical formulas were verified against the source paper. Explanations were rewritten where they were inaccurate, overly generic, or insufficiently deep.

Step 6: Slide Design. The 15 slides were designed with a consistent dark tech aesthetic. Each slide was purpose-built around a single concept, using architecture diagrams, annotated formulas, and comparison tables.

Step 7: Website Development. The interactive website was built and deployed on Netlify, featuring animated attention visualizations, a clickable architecture diagram, a step-by-step processing demo, and an interactive quiz.

Step 8: MCQ Bank. Questions were written to span all Bloom’s taxonomy levels—from recall to application and analysis—ensuring comprehensive assessment of understanding.

Step 9: Testing. The website and slides were tested by team members and peers to check for accuracy, usability, and clarity before final submission.

6. Challenges Faced and How We Managed Them

The following challenges were encountered during the project and addressed as described.

6.1. Challenge 1: Depth vs. Accessibility Trade-off

Problem: The Transformer architecture is mathematically rich. Explaining the full formulation (multi-head attention dimensions, masking matrices, positional encoding equations) risks overwhelming a non-expert audience, while oversimplifying it fails to demonstrate deep understanding.

Solution: We adopted a layered explanation strategy. Slides present intuitive visual explanations first, with formal mathematics as secondary annotations. The website provides both high-level narratives and expandable technical detail. This “zoom in/zoom out” approach ensures accessibility without sacrificing depth.

6.2. Challenge 2: Verifying AI-Generated Content

Problem: When using LLMs to draft explanations or generate MCQ questions, some outputs contained subtle inaccuracies or overly simplified statements that could mislead students.

Solution: Every AI-generated statement was cross-checked against the original paper and trusted secondary sources. Mathematical formulas were verified by hand. Questions where the AI-provided reasoning was unclear or ambiguous were either rewritten or discarded.

6.3. Challenge 3: Making Self-Attention Interactive

Problem: The attention mechanism is inherently a matrix-level operation. Visualizing it interactively in a web interface—in a way that is both accurate and intuitive—required significant technical effort.

Solution: We simplified the visualization to show attention weights as color-coded connections between words, allowing users to click a word and see which others it attends to most strongly. While this does not show all mathematical details, it accurately conveys the core intuition of the mechanism.

6.4. Challenge 4: Distinguishing Encoder-Only, Decoder-Only, and Encoder-Decoder Models

Problem: The term “Transformer” is used to refer to three distinct families of models (BERT-style encoder-only, GPT-style decoder-only, T5-style encoder-decoder), which caused confusion during the explanation design phase.

Solution: We clearly scoped the project to the original encoder-decoder architecture as presented in the 2017 paper, and explicitly noted in the presentation and website where modern variants differ (e.g., that GPT uses a decoder-only architecture, and BERT uses an encoder-only).

6.5. Challenge 5: Designing Meaningful MCQs at Multiple Levels

Problem: It was easy to write trivial recall questions (“What does Q stand for in QKV?”) but harder to write questions that tested genuine understanding and application.

Solution: We deliberately drafted questions at multiple cognitive levels: recall, comprehension, application, and analysis. For example, rather than asking the formula for attention, we asked why the scaling factor $\frac{1}{\sqrt{d_k}}$ is necessary—testing conceptual understanding rather than memorization.

6.6. Challenge 6: Lack of Web Development and UI Design Knowledge

Problem: Building an interactive educational website requires skills in front-end web development (HTML, CSS, JavaScript), UI/UX design, and deployment—areas in which the team had little to no prior experience. Producing a polished, responsive, and functional site from scratch posed a significant barrier, particularly for features such as animated attention visualizations and the interactive quiz.

Solution: We leveraged AI code generation tools to produce initial scaffolding for the website’s components, then iteratively refined the output by testing it in the browser and fixing errors. Online documentation (MDN Web Docs, CSS Tricks) and tutorials were consulted to understand the generated code well enough to modify and debug it. Deployment was handled through Netlify’s drag-and-drop interface, which removed the need to configure server infrastructure. While the final product does not reflect expert-level front-end engineering, this approach allowed us to ship a functional and visually coherent demo within the project timeline.

6.7. Challenge 7: Maintaining Pedagogical Coherence Across Multiple Deliverables

Problem: With three separate deliverables—slides, a website, and an MCQ bank—produced in parallel by different team members, there was a risk of inconsistency in terminology, level of detail, and conceptual framing. For instance, one deliverable might describe the attention scaling factor differently from another, or use different notation for the same variable.

Solution: We established a shared terminology reference document at the start of the project, listing agreed-upon notation, definitions, and scope boundaries (e.g., always using d_k for the key dimension, always referring to the 2017 encoder-decoder as “the original Transformer”). Regular cross-review sessions between team members ensured that the three deliverables remained aligned and that a student moving from the slides to the website to the quiz would encounter a consistent learning experience.

6.8. Challenge 8: Controlling the Tone and Register of AI-Generated Explanations

Problem: LLMs tend to produce explanations that are either too formal and dense (resembling academic writing) or too conversational and imprecise (omitting necessary technical nuance). Neither extreme is appropriate for an educational presentation targeting engineering students with a foundational background in neural networks.

Solution: We developed a consistent prompting strategy for content generation, specifying the target audience, the desired level of prior knowledge assumed, and the

approximate length and tone for each section. Generated drafts were then edited to match the established style guide: precise but approachable, with concrete analogies alongside formal definitions. This iterative prompt-then-edit workflow significantly reduced the time spent on rewrites compared to generating content without explicit constraints.

6.9. Challenge 9: Avoiding Hallucinated Citations and Fabricated Claims

Problem: When asking LLMs to provide background context or supporting references, the models occasionally produced plausible-sounding but fabricated citations—incorrect author names, non-existent paper titles, or misattributed findings. Including such errors in an academic report or educational material would undermine credibility and mislead readers.

Solution: A strict policy was adopted: no citation or empirical claim was included in any deliverable unless it could be independently verified against a primary source. All references in this report were located and read by at least one team member before inclusion. AI tools were used solely to *suggest* candidate sources, never to serve as the authoritative source themselves.

7. AI Tools Usage Documentation

Per the course requirements, we document how AI tools were used throughout the project:

Table 2: AI tool usage in the project

Tool Used	Task	Human Contribution
LLM (Claude)	Drafting slide explanations, website copy, and initial MCQ pool	Reviewed, edited, and verified all outputs
LLM (Claude)	Generating initial website component code	Reviewed code, tested functionality, and debugged issues
AI Design Tools	Suggesting slide layouts and color schemes	Final design decisions and content placement
LLM (Claude)	Drafting initial versions of this report	Rewrote sections, verified technical accuracy, and added original analysis

In all cases, the team took responsibility for the final output. No AI-generated content was used “as-is.”

8. Conclusions

This project achieved its core aim: to deliver a clear, accurate, visually compelling, and interactive explanation of the Transformer architecture using AI tools responsibly.

The Transformer represents one of the most significant architectural innovations in the history of deep learning. Its ability to model arbitrary-length dependencies in a single parallelizable operation—through the elegant mechanism of scaled dot-product attention—fundamentally changed what was possible in NLP and beyond.

Through this project, we drew the following key conclusions:

- 1. Attention is all you need—for sequence modeling.** The core insight of replacing recurrence with attention proved transformative. By allowing every token to directly interact with every other token, the Transformer overcame the fundamental bottlenecks of sequential architectures.
- 2. Multi-head attention is more expressive than single attention.** Running multiple attention heads in parallel allows the model to simultaneously capture different types of relationships (syntactic, semantic, positional), leading to richer representations.
- 3. No architecture is without trade-offs.** The $O(n^2)$ complexity of self-attention, the need for massive data and compute, and the lack of true reasoning capability are genuine limitations. Understanding when to use Transformers—and when to consider alternatives—is as important as understanding how they work.
- 4. AI tools accelerate learning, but do not replace understanding.** Using LLMs and code generation tools significantly accelerated our workflow, but every output required careful review. The project reinforced that AI tools are most powerful when combined with genuine domain knowledge.
- 5. Visualization is key to understanding.** The most impactful part of our project—both for ourselves and for our audience—was the interactive attention visualization on the website. Seeing attention weights update in real time when clicking on a word made the mechanism intuitively clear in a way that formulas alone cannot achieve.